Offline Reinforcement Learning

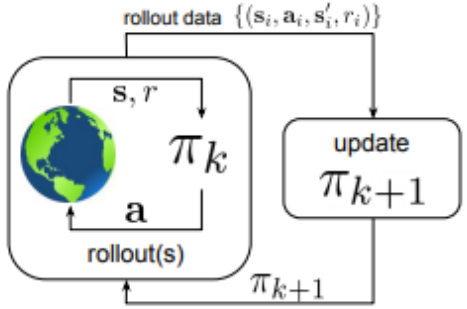
CS 185/285

Instructor: Sergey Levine
UC Berkeley

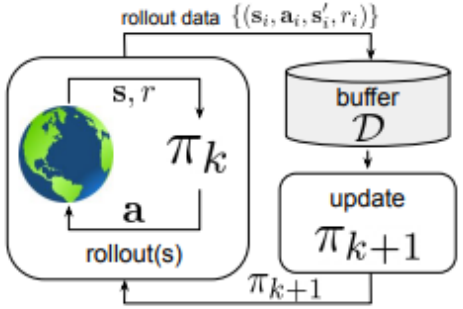
Part 1:
What is offline reinforcement learning?

Offline reinforcement learning

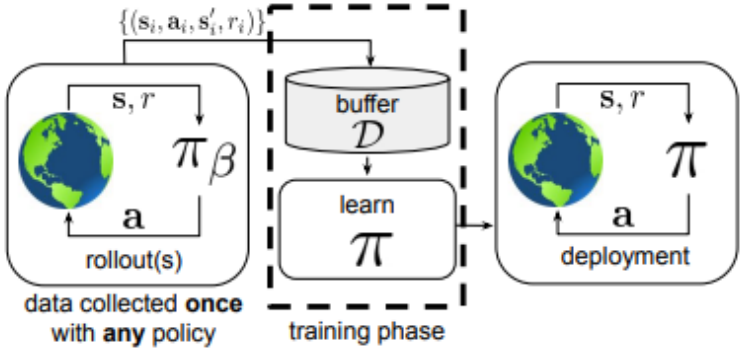
on-policy RL



off-policy RL

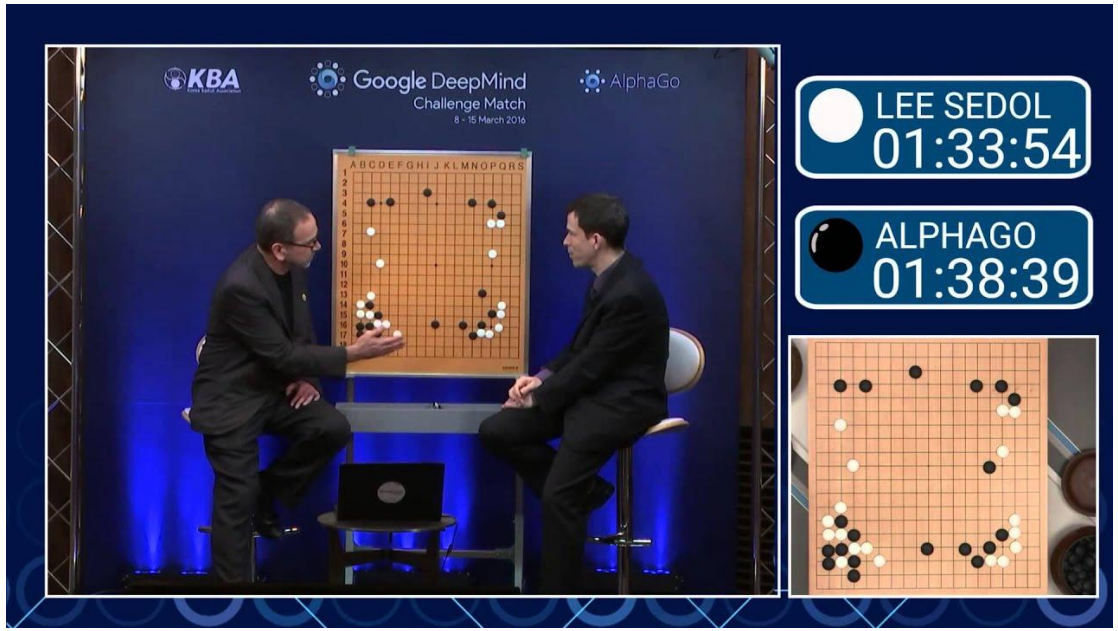


offline reinforcement learning



Why?

Impressive because no person had thought of it!



"Move 37" in Lee Sedol AlphaGo match: reinforcement learning "discovers" a move that surprises everyone

Impressive because it looks like something a person might draw!

So where does that leave us?

Data-Driven AI

All about using **data**

+ learns about the real world from data

- doesn't try to do **better** than the data

Data without **optimization** doesn't allow us to solve new problems in new ways

Reinforcement Learning

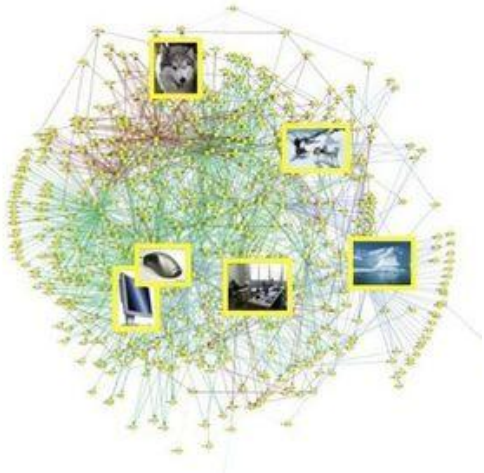
All about **optimization**

+ optimizes a goal with emergent behavior

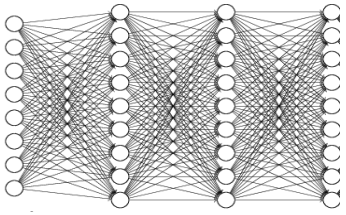
- doesn't make use of real-world data

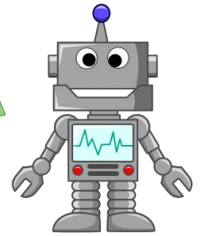
Optimization without **data** is hard to apply to the real world outside of simulators

The big picture

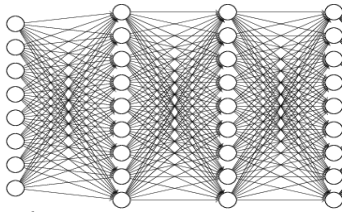


offline training





fast, safe online RL finetuning



offline RL gives us one way to use prior data for RL training

understanding offline RL also really helps us to understand RL more generally

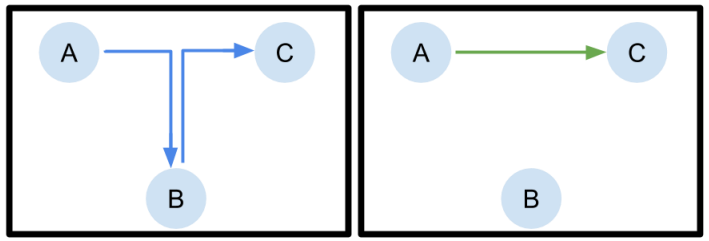
it is not the **only** way to pretrain for RL (e.g., LLMs are pretrained with imitation learning!)

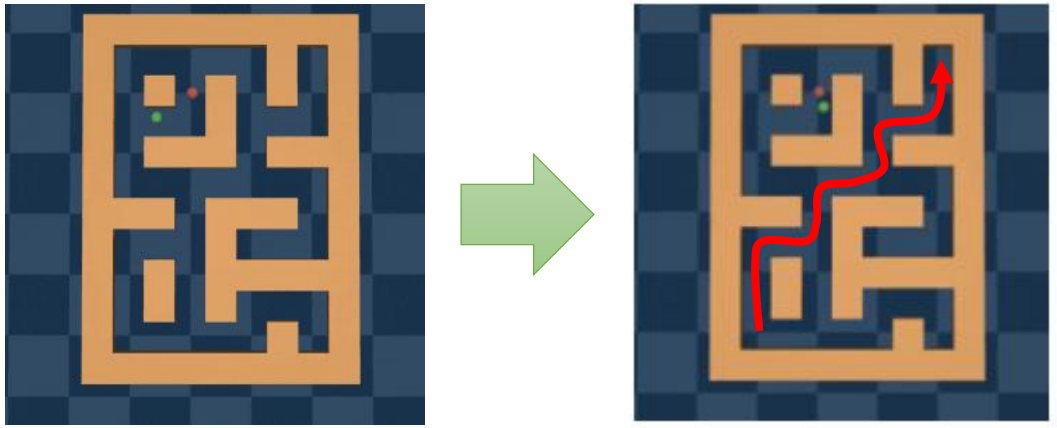
How is this even possible?

1. Find the “good stuff” in a dataset full of good and bad behaviors

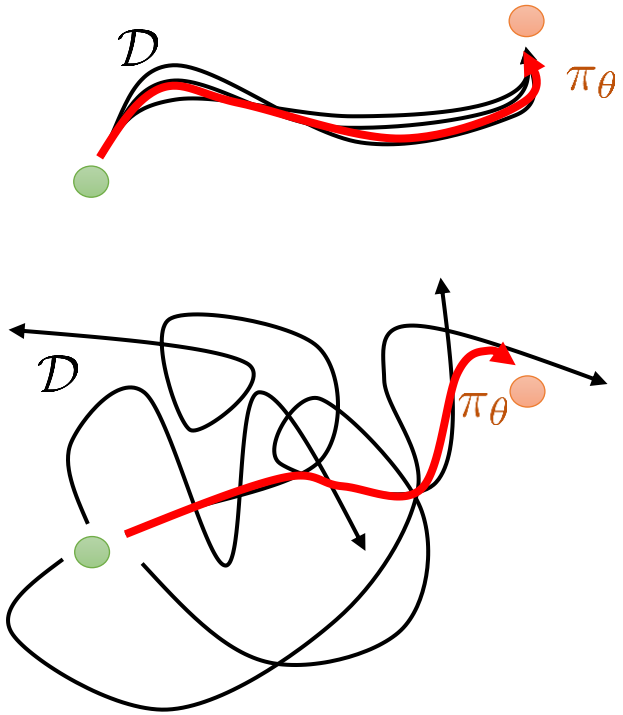
2. Generalization: good behavior in one place may suggest good behavior in another place

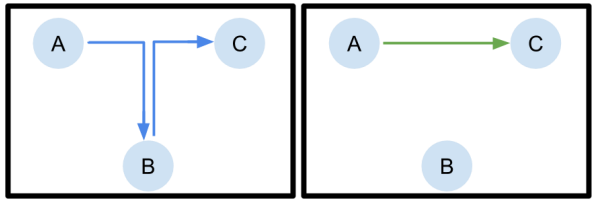
3. “Stitching”: parts of good behaviors can be recombined





What do we expect offline RL methods to do?

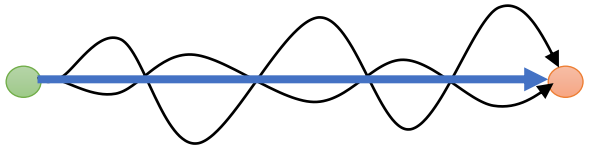




"Macro-scale" stitching

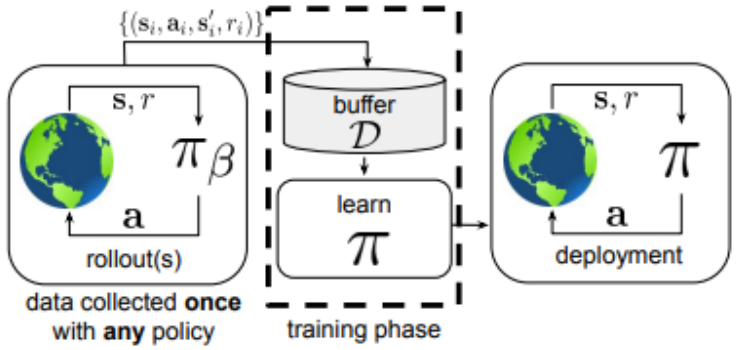
But this is just the clearest example!

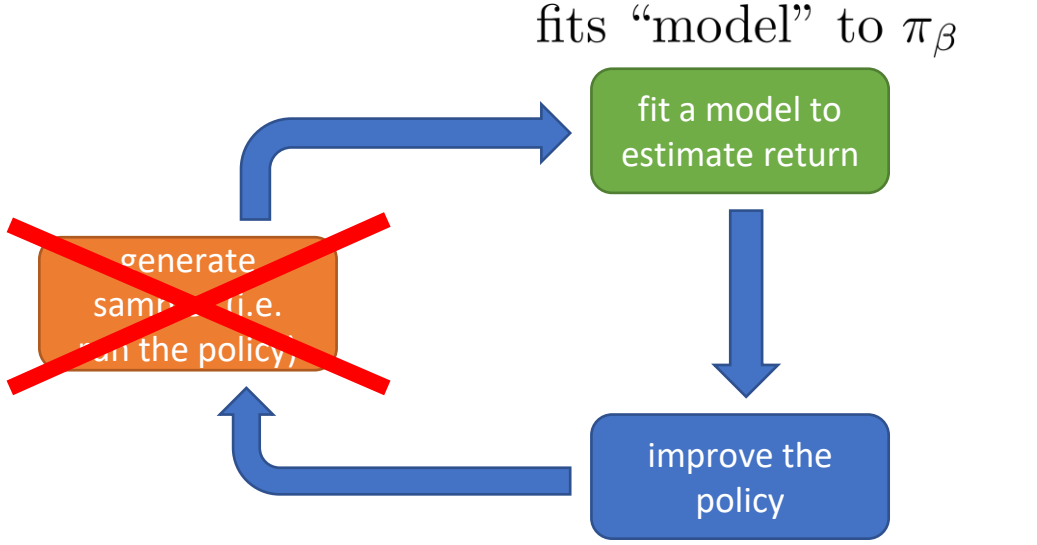
"Micro-scale" stitching:



Part 2:
Distributional shift

The basic challenge with offline RL





Formally:

$$\mathcal{D} = \{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}_{i=1}^N$$

$$\mathbf{s} \sim p_\beta(\mathbf{s})$$

$$\mathbf{a} \sim \pi_\beta(\mathbf{a}|\mathbf{s})$$ ← generally **not** known

$$\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s}, \mathbf{a})$$

RL objective: $\max_\theta \sum_{t=0}^{H} E_{\mathbf{s}_t, \mathbf{a}_t \sim \pi_\theta(\mathbf{a}|\mathbf{s})}[\gamma^t r(\mathbf{s}_t, \mathbf{a}_t)]$

model is valid under $\mathbf{s}, \mathbf{a} \sim \pi_\beta(\mathbf{s}, \mathbf{a})$

not valid under $\mathbf{s}, \mathbf{a} \sim \pi_\theta(\mathbf{s}, \mathbf{a})$

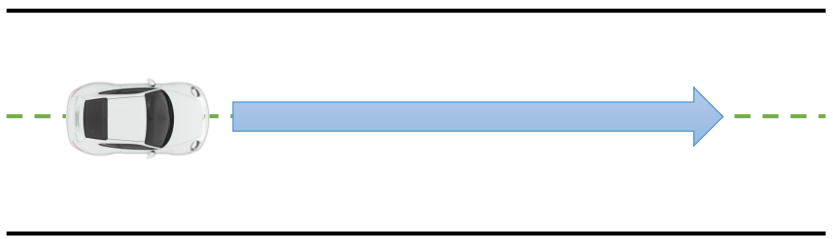
model could be Q-function $Q^{\pi_\theta}(\mathbf{s}, \mathbf{a})$

model could be dynamics $\hat{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$

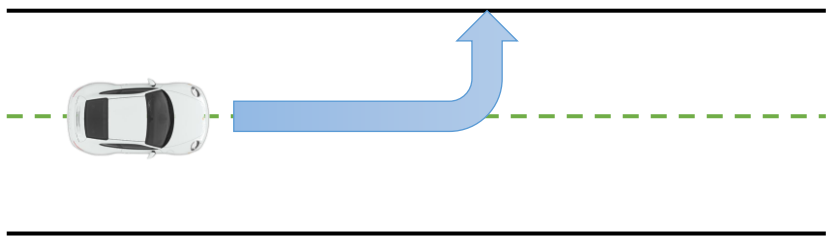
An example

Fundamental problem: counterfactual queries

Training data



What the policy wants to do



Is this good? Bad?

How do we know if we didn't see it in the data?

Online RL algorithms don't have to handle this, because they can simply **try** this action and see what happens

Offline RL methods must somehow account for these unseen ("out-of-distribution") actions, ideally in a safe way

…while still making use of generalization to come up with behaviors that are better than the best thing seen in the data!

Revisiting distributional shift

Definition: we train $p_\theta(y|\mathbf{x})$ on $\mathbf{x} \sim p_{\text{train}}(\mathbf{x})$

we test $p_\theta(y|\mathbf{x})$ on $\mathbf{x} \sim p_{\text{test}}(\mathbf{x})$

$$p_{\text{test}}(\mathbf{x}) \neq p_{\text{train}}(\mathbf{x})$$

How bad could it be?

Imagine you prepare for a math exam, and get an exam on ancient Greek literature

More precisely…

Example empirical risk minimization (ERM) problem:

a.k.a. standard maximum likelihood estimation

$$\theta \leftarrow \arg\min_{\theta} E_{\mathbf{x}\sim p(\mathbf{x}),y\sim p(y|\mathbf{x})}\left[(f_\theta(\mathbf{x})-y)^2\right]$$

given some $\mathbf{x}^\star$, is $f_\theta(\mathbf{x}^\star)$ correct?

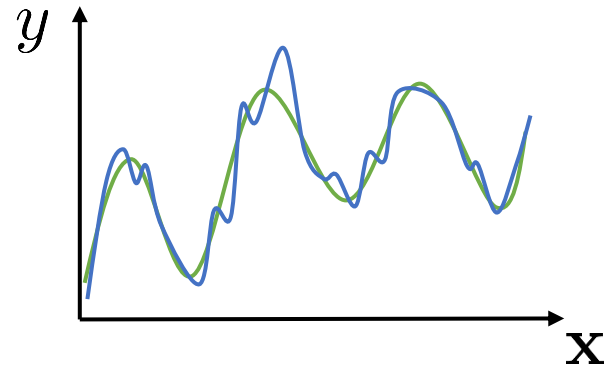
$E_{\mathbf{x}\sim p(\mathbf{x}),y\sim p(y|\mathbf{x})}\left[(f_\theta(\mathbf{x})-y)^2\right]$ is low

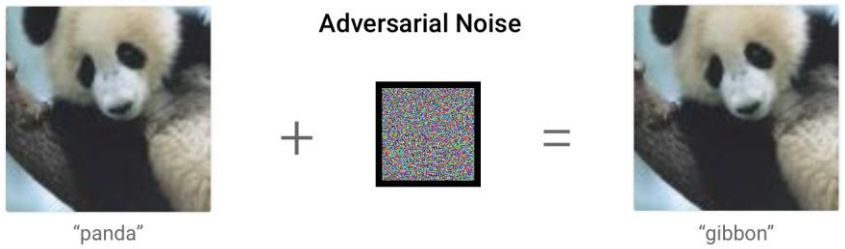
$E_{\mathbf{x}\sim \bar{p}(\mathbf{x}),y\sim p(y|\mathbf{x})}\left[(f_\theta(\mathbf{x})-y)^2\right]$ is not, for general $\bar{p}(\mathbf{x}) \neq p(\mathbf{x})$

what if $\mathbf{x}^\star \sim p(\mathbf{x})$? not necessarily...

usually we are not worried – neural nets generalize well!

what if we pick $\mathbf{x}^\star \leftarrow \arg\max_{\mathbf{x}} f_\theta(\mathbf{x})$?





Where does it appear for each off-policy RL method?

Q-learning & Q-function actor-critic

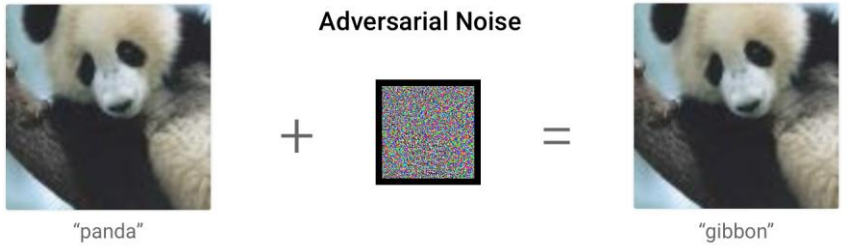
$$\phi \leftarrow \arg\min_{\phi} \mathbb{E}_{\mathbf{s},\mathbf{a}\sim\pi_\beta(\mathbf{s},\mathbf{a})}\|Q_\phi^{\pi_\theta}(\mathbf{s},\mathbf{a}) - y\|^2 \qquad y_i = r(\mathbf{s}_i,\mathbf{a}_i) + \gamma\mathbb{E}_{\mathbf{a}'\sim\pi_\theta(\mathbf{a}'|\mathbf{s}'_i)}[Q_\phi^{\pi_\theta}(\mathbf{s}'_i,\mathbf{a}')]$$

because we only have $\mathbf{s}'_i$ for $\mathbf{s}_i, \mathbf{a}_i$

$(\mathbf{s}_i, \mathbf{a}_i) \sim \pi_\beta(\mathbf{s}, \mathbf{a})$

$$\pi_\theta(\mathbf{a}|\mathbf{s}) \neq \pi_\beta(\mathbf{a}|\mathbf{s})$$

$$\text{worse: } \theta \leftarrow \arg\max_\theta \mathbb{E}_{\mathbf{a}\sim\pi_\theta(\mathbf{a}|\mathbf{s})}[Q_\phi^{\pi_\theta}(\mathbf{s},\mathbf{a})]$$



Where does it appear for each off-policy RL method?

model-based RL

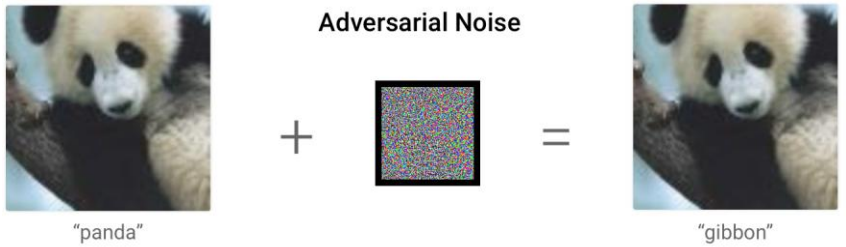
$$f \leftarrow \arg\min_f \mathbb{E}_{\mathbf{s},\mathbf{a}\sim\pi_\beta(\mathbf{s},\mathbf{a}),\mathbf{s}'\sim p(\mathbf{s}'|\mathbf{s},\mathbf{a})}||f(\mathbf{s},\mathbf{a})-\mathbf{s}'||^2$$

$\mathbb{E}_{\mathbf{s},\mathbf{a}\sim\pi_\beta(\mathbf{s},\mathbf{a}),\mathbf{s}'\sim p(\mathbf{s}'|\mathbf{s},\mathbf{a})}||f(\mathbf{s},\mathbf{a})-\mathbf{s}'||^2$ is low

$\mathbb{E}_{\mathbf{s},\mathbf{a}\sim\pi_\theta(\mathbf{s},\mathbf{a}),\mathbf{s}'\sim p(\mathbf{s}'|\mathbf{s},\mathbf{a})}||f(\mathbf{s},\mathbf{a})-\mathbf{s}'||^2$ is not, for general $\pi_\theta \neq \pi_\beta$

worse:

we pick π_θ to *maximize* the reward under f!



Where does it appear for each off-policy RL method?

off-policy policy gradient (importance-sampled surrogate):

$$J(\theta) = \sum_{i=1}^{N} \sum_{t=1}^{H} \underbrace{\frac{\pi_\theta(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})}{\pi_\beta(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})}} A^{\pi_\theta}(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \beta \underbrace{\log \pi_\theta(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})}$$

ratio very far from 1.0 = exploding variance

"just don't go there"

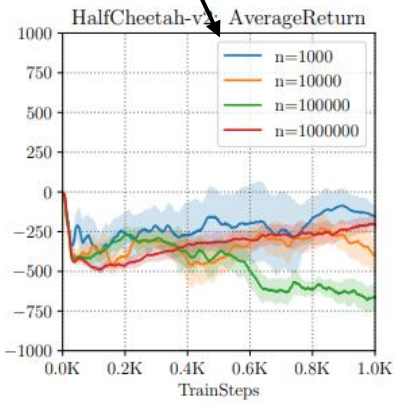
This is an example of a *policy constraint*

the answer to counterfactual questions has "high variance" (depends on whether you saw it or not!)

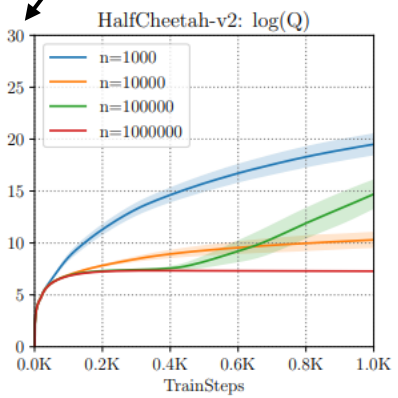
An example with soft actor-critic

amount of data

log scale (massive overestimation)



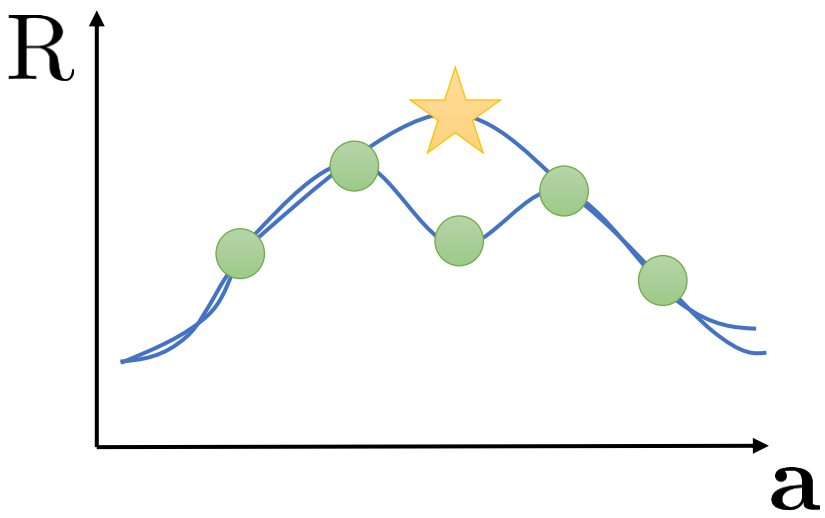
how well it does



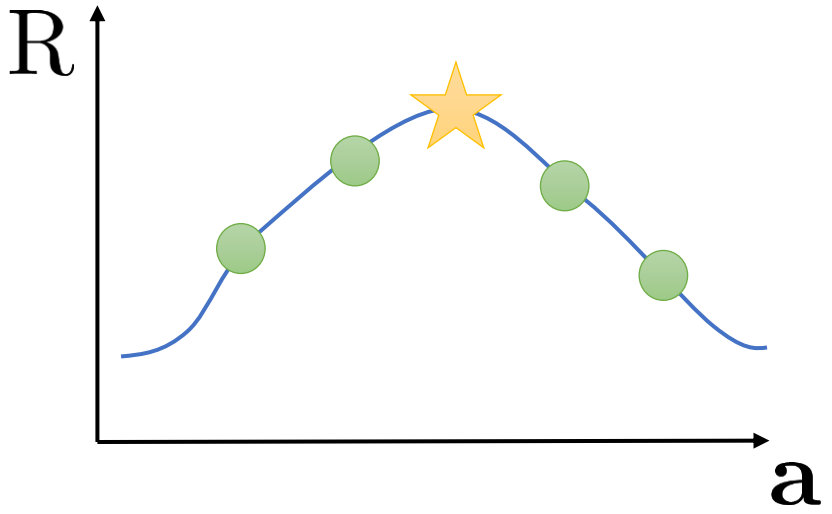
how well it *thinks* it does (Q-values)

Issues with generalization are not corrected

online RL setting



offline RL setting



existing challenges with sampling error and function approximation error in standard RL become **much more severe** in offline RL

Part 3:
Policy constraints

Some principles for **offline** RL

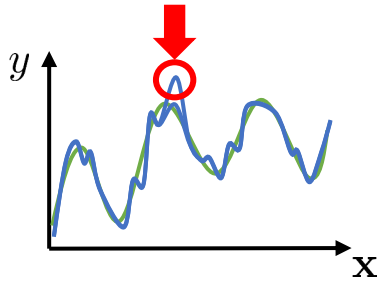
- Many different methods, similar principles seem to be effective:

use value-based methods (i.e., Q-learning or Q-function actor-critic)

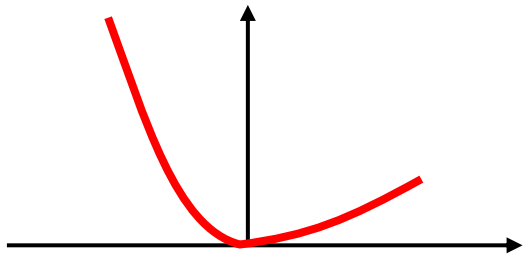
somehow fix the distributional shift problem

$$D_{\mathrm{KL}}(\pi(\mathbf{a}|\mathbf{s})\|\pi_\beta(\mathbf{a}|\mathbf{s})) \leq \epsilon$$

policy constraints



pessimism (e.g., CQL)



avoid OOD actions in updates (e.g., IQL)

train the policy so that it stays close to data

train the Q-function so that it doesn't overestimate

set up the whole algorithm to avoid out-of-sample actions

Policy constraints

Recall:

off-policy policy gradient (importance-sampled surrogate):

$$J(\theta) = \sum_{i=1}^{N} \sum_{t=1}^{H} \frac{\pi_\theta(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})}{\pi_\beta(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})} A^{\pi_\theta}(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \beta \log \pi_\theta(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})$$

"just don't go there"

$$\mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a}_t|\mathbf{s}_t^{(i)})}[Q^\pi(\mathbf{s}_t^{(i)}, \mathbf{a}_t)]$$

Why?

$$D_{\mathrm{KL}}(\pi_\beta(\mathbf{a}|\mathbf{s}_t^{(i)}) \| \pi_\theta(\mathbf{a}|\mathbf{s}_t^{(i)}))$$ (up to a constant)

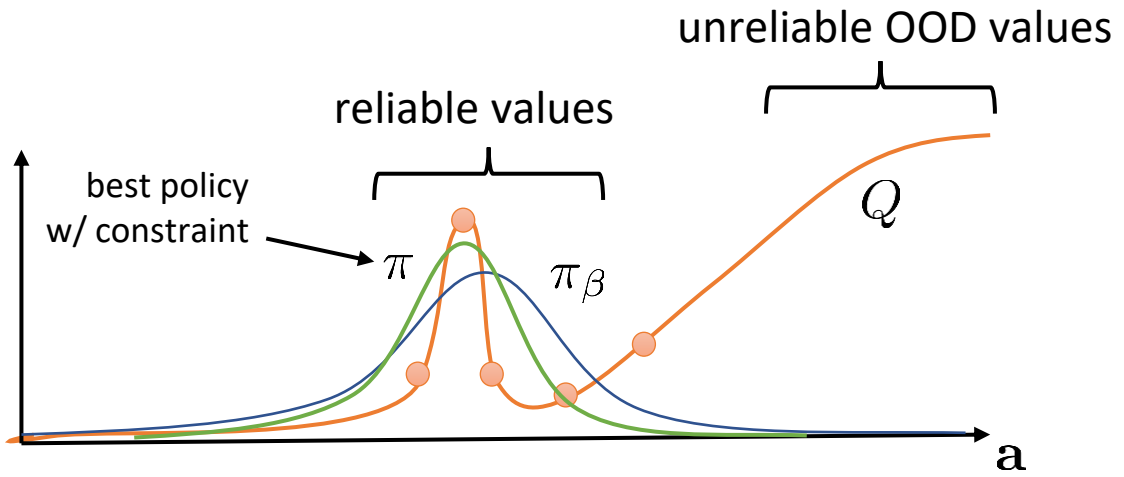
could also use $D_{\mathrm{KL}}(\pi_\theta(\mathbf{a}|\mathbf{s}_t^{(i)}) \| \pi_\beta(\mathbf{a}|\mathbf{s}_t^{(i)}))$

Why does a policy constraint fix the problem?

$$\phi \leftarrow \arg\min_{\phi} \mathbb{E}_{\mathbf{s},\mathbf{a}\sim\pi_\beta(\mathbf{s},\mathbf{a})}\|Q_\phi^{\pi_\theta}(\mathbf{s},\mathbf{a}) - y\|^2$$

$$y_i = r(\mathbf{s}_i,\mathbf{a}_i) + \gamma\mathbb{E}_{\mathbf{a}'\sim\pi_\theta(\mathbf{a}'|\mathbf{s}'_i)}[Q_\phi^{\pi_\theta}(\mathbf{s}'_i,\mathbf{a}')]$$

$$\pi_\theta(\mathbf{a}|\mathbf{s}) \neq \pi_\beta(\mathbf{a}|\mathbf{s})$$



keeping π_θ "close" to π_β limits distributional shift

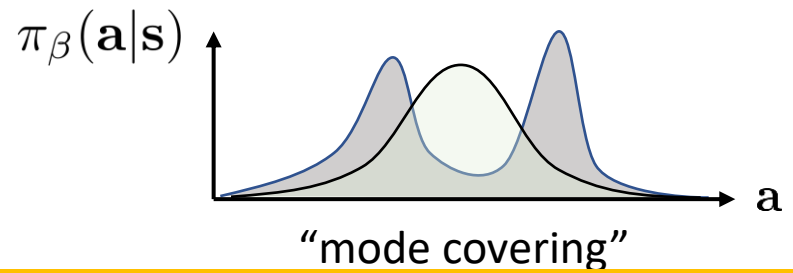
What kind of constraint do we use?

Aside: forward KL and reverse KL

$$D_{\mathrm{KL}}(\pi_\beta(\mathbf{a}|\mathbf{s})\|\pi_\theta(\mathbf{a}|\mathbf{s})) = -\mathbb{E}_{\mathbf{a}\sim\pi_\beta(\mathbf{a}|\mathbf{s})}[\log \pi_\theta(\mathbf{a}|\mathbf{s})] + \text{const}$$

"forward KL"

giving probability of 0 to any sample
leads to a divergence of ∞



"mode covering"

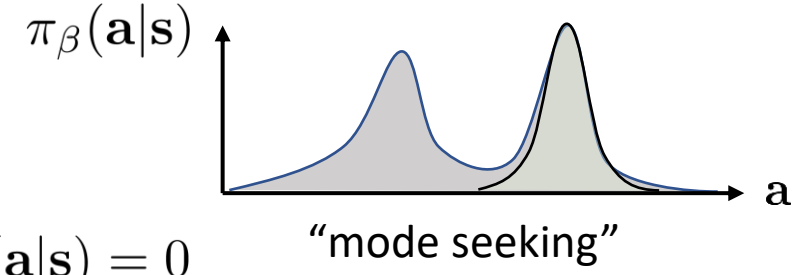
why would we want this?

$$D_{\mathrm{KL}}(\pi_\theta(\mathbf{a}|\mathbf{s})\|\pi_\beta(\mathbf{a}|\mathbf{s})) = -\mathbb{E}_{\mathbf{a}\sim\pi_\theta(\mathbf{a}|\mathbf{s})}[\log \pi_\beta(\mathbf{a}|\mathbf{s})] - \mathcal{H}(\pi_\theta(\mathbf{a}|\mathbf{s})$$

"reverse KL"

just like adding $\log \pi_\beta(\mathbf{a}|\mathbf{s})$ to reward
and using maximum entropy RL

huge penalty for non-zero prob to any sample where $\pi_\beta(\mathbf{a}|\mathbf{s}) = 0$



"mode seeking"

why would we want this?

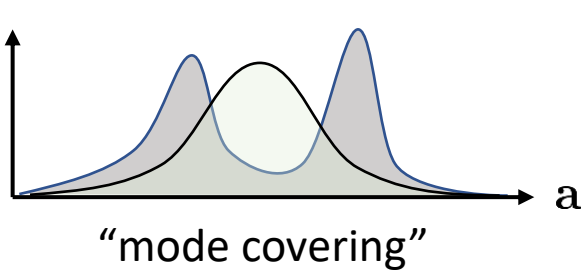
What kind of constraint do we use?

Aside: forward KL and reverse KL

$$D_{\mathrm{KL}}(\pi_\beta(\mathbf{a}|\mathbf{s})\|\pi_\theta(\mathbf{a}|\mathbf{s})) = -\mathbb{E}_{\mathbf{a}\sim\pi_\beta(\mathbf{a}|\mathbf{s})}[\log \pi_\theta(\mathbf{a}|\mathbf{s})] + \text{const}$$

"forward KL"

good if we don't know $\pi_\beta(\mathbf{a}|\mathbf{s})$ (reuse samples)
good if we don't want to lose modes (e.g., LLM)
popular with PPO and LLMs

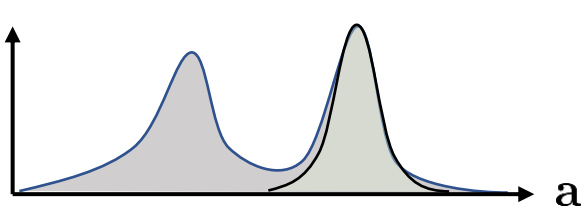


"mode covering"

$$D_{\mathrm{KL}}(\pi_\theta(\mathbf{a}|\mathbf{s})\|\pi_\beta(\mathbf{a}|\mathbf{s})) = -\mathbb{E}_{\mathbf{a}\sim\pi_\theta(\mathbf{a}|\mathbf{s})}[\log \pi_\beta(\mathbf{a}|\mathbf{s})] - \mathcal{H}(\pi_\theta(\mathbf{a}|\mathbf{s}))$$

"reverse KL"

good if you want "best thing inside the data"
good if you use MaxEnt RL (just add $\log \pi_\beta$ to r)
requires estimating $\pi_\beta(\mathbf{a}|\mathbf{s})$



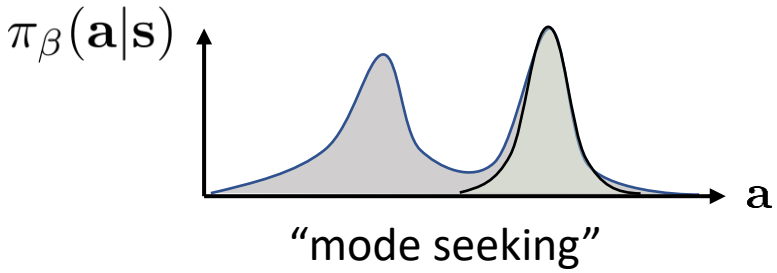
"mode seeking"

What is the “ideal” constraint?

$$D_{\mathrm{KL}}(\pi_\theta(\mathbf{a}|\mathbf{s})\|\pi_\beta(\mathbf{a}|\mathbf{s}))$$

“reverse KL”

we like this because we are not forced to “keep” bad modes with low reward



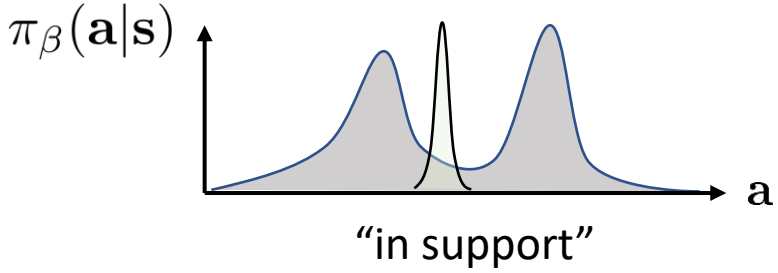
“mode seeking”

$$D_{\mathrm{support}}(\pi_\theta(\mathbf{a}|\mathbf{s}), \pi_\beta(\mathbf{a}|\mathbf{s})) = \mathbb{E}_{\mathbf{a}\sim\pi_\theta(\mathbf{a}|\mathbf{s})}[\delta(\pi_\beta(\mathbf{a}|\mathbf{s}) = 0)]$$

only pay the penalty for actions that really can’t happen under the behavior policy

but allow any behavior that can happen in the data

very hard to approximate tractably



“in support”